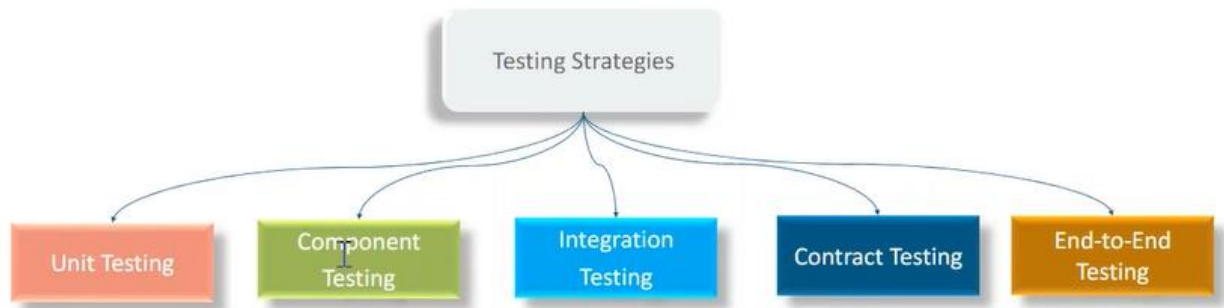


MicroServices Testing:

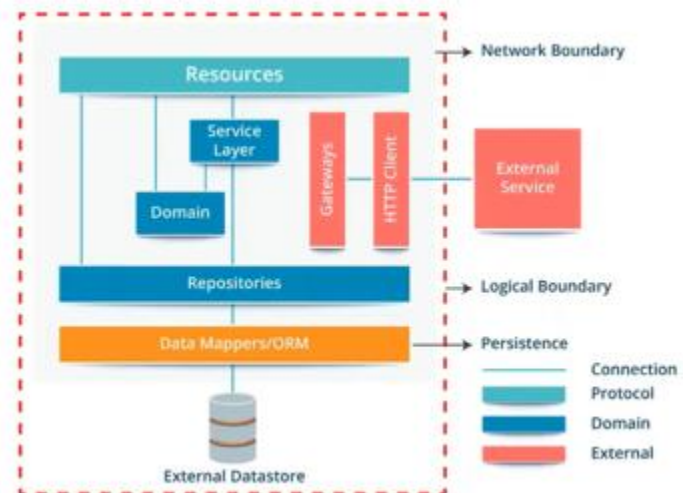
Testing Strategies:



What are the different layers in ms architecture, and in each layer which testing?

microservices echo system

- Resource – Resource, Service & Domain
- Communication – Gateway, Clients, External Services
- Data Access – External Database & Sources

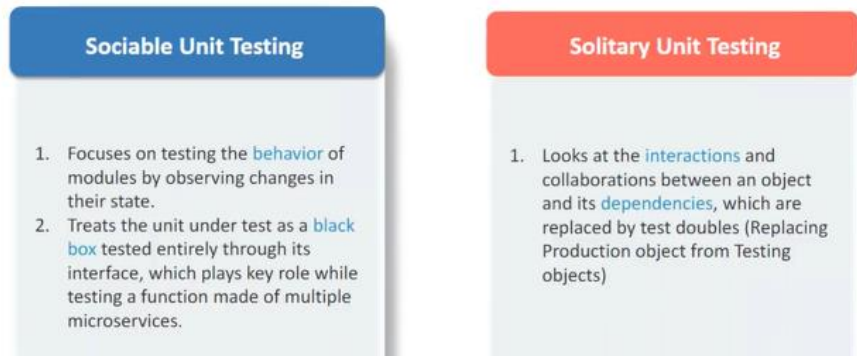


Unit Testing:

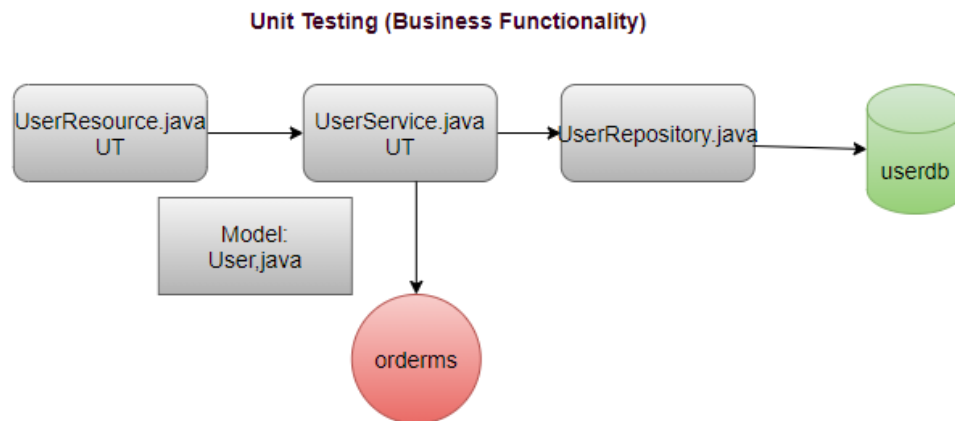
Unit testing main purpose is to test the business functionality, it's not supposed to test the contract with other ms or integration with DB.

Unit testing have maximum volume (number of unit test).

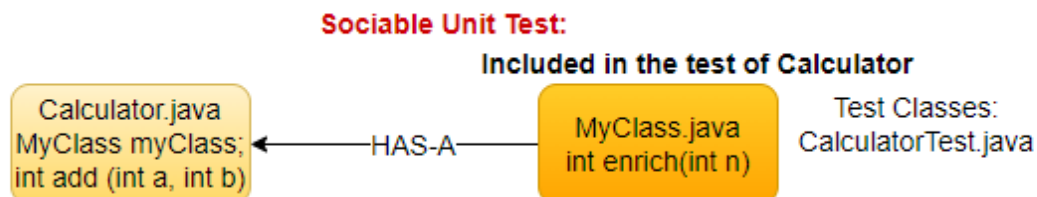
Microservices unit testing generally use the two styles given below. They are not competing styles and compliment each other while solving different testing problems.



Unit- Smallest testable code our service, test locally, it may call other class locally (sociable), but no network call. Unit testing is just about to test the business logic (mostly written for service class), being part of the microservice. Any integration with other microservice or database comes under integration boundary which comes under integration testing, not comes under the unit testing.

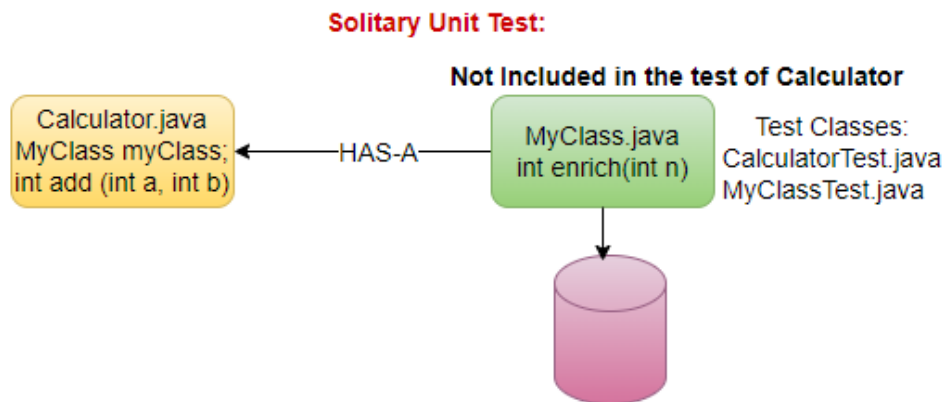


Sociable Testing:



All dependent classes should also be tested, e.g., if we are testing calculator class then we have to also test all its dependent classes.

Solitary Testing:



Just focus on one class, e.g., if we are testing calculator class then just focus on calculator class, don't test all its dependent classes (using **Mockito**) Used only for testing purpose.

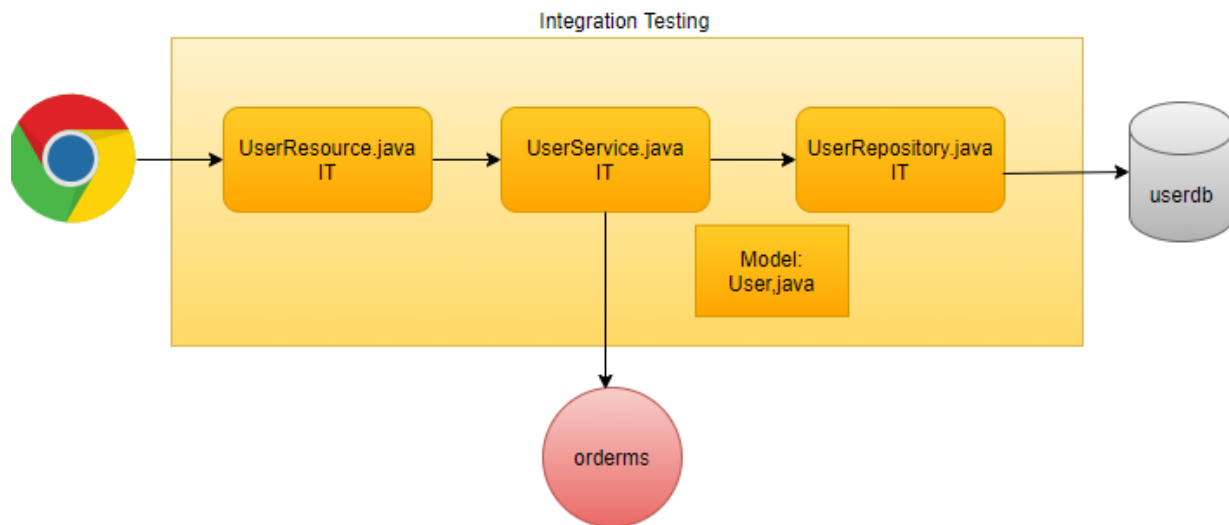
Sociable/Solitary/Unit Test:

```
class Calculator{  
    MyClass myClass;  
  
    int add(int a, int b) {  
        int c = myClass.enrich(a);  
        int d = myClass.enrich(b);  
        # return a+b;  
        return c + d;  
    }  
}
```

```
class MyClass {  
    int enrich(int n) {  
        n + 2  
    }  
}
```

Integration Testing (w.r.t single ms)

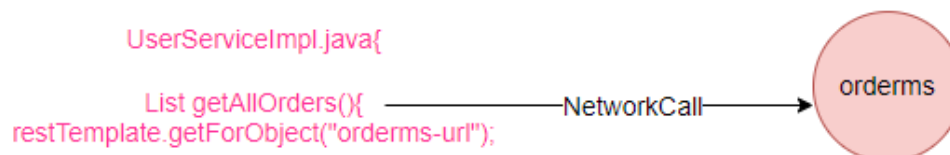
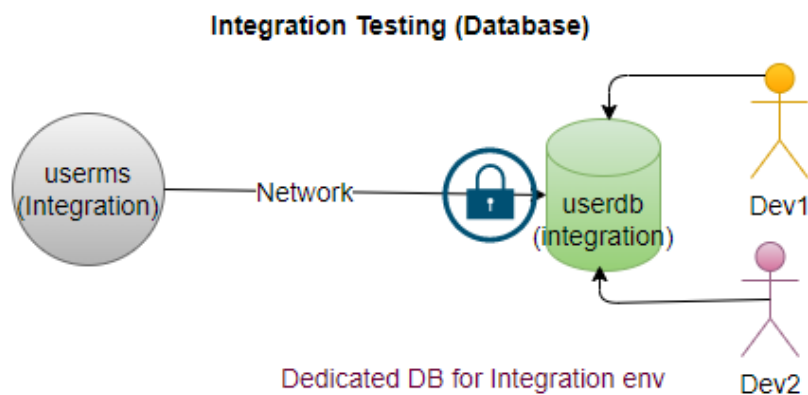
Focus on to test the Integration boundaries rather than to test the business logic, business logic intensively tested on unit test



API testing: between chrome and UserResource

Contract testing: UserService and ordersms

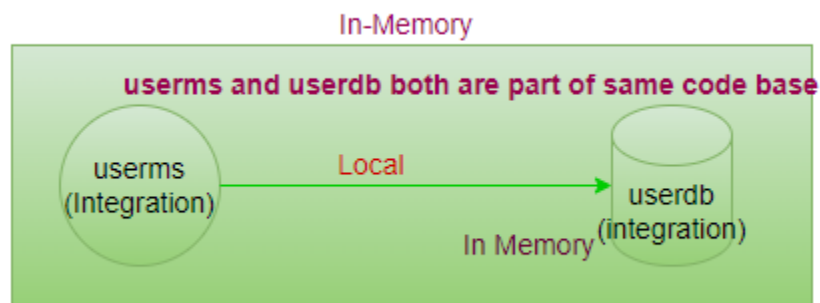
Integration Test- Database



- Since All dev, update on same integration, DB may corrupt. In case of any issue for infra team SLA of non-production environment is much higher.

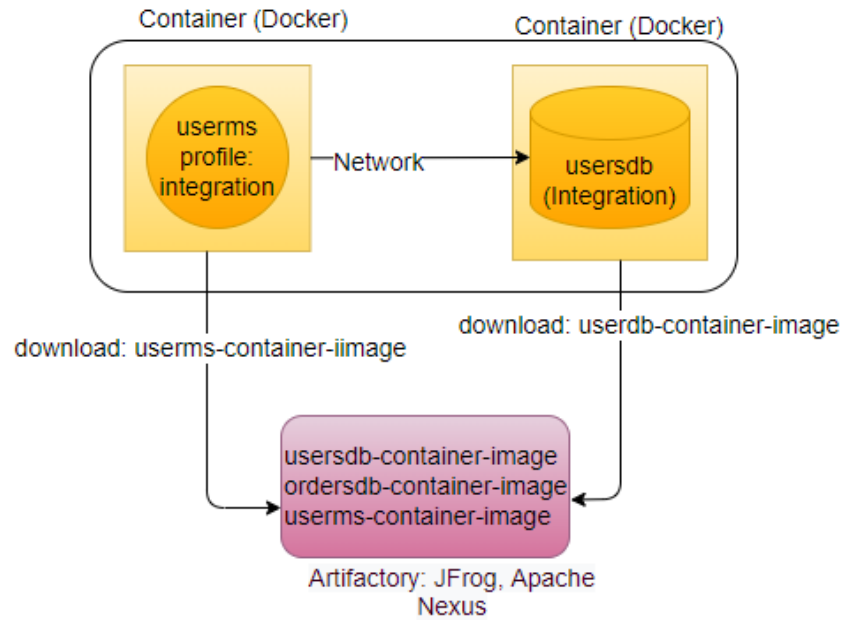
- Some user may keep putting locks on DB.
- Separate DBA, ensure data is clean.
- Its dev data may corrupt the data.
- Testing is block
- Very close to production environment.

UC2: In-memory DB

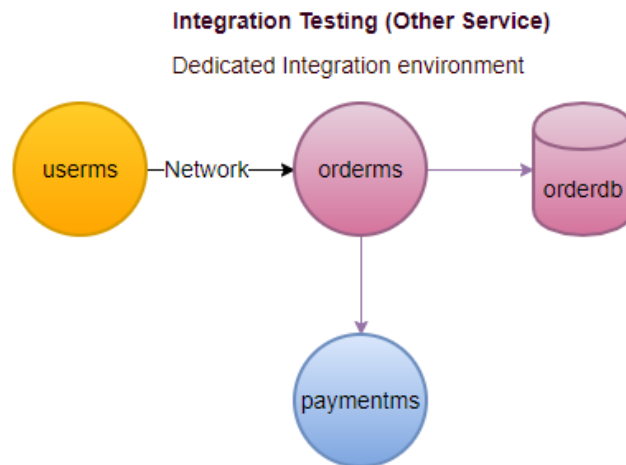


- Can't use for performance testing.
- Not available for all vendors (limited sources)
- Not suitable for production like load.
- Integration is just like local process.
- It's always available, this will be part of your code
- No Issue of data corruption
- Very fast

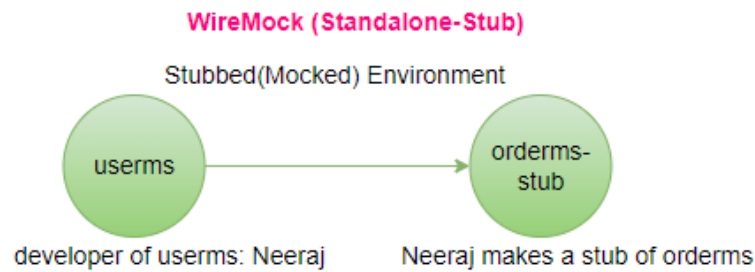
UC3: container DB



Integration Test- Other Service (Dedicated Environment)



Integration Test- Other Service (Stubbed-Mocked Environment)



As a developer of users, create our own stub of orders-stub for testing purpose, it doesn't contain any business logic of orders just simple request and response.

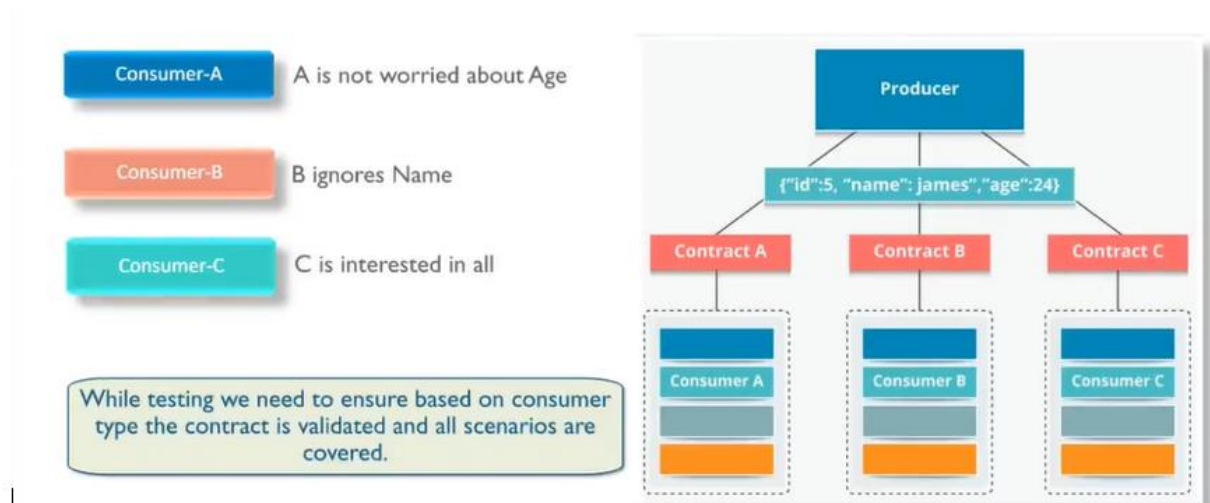
stub is nothing but standalone service, where we put the static responses for all the possible calls, which are expecting from users (base on request return the static response). WireMock help us to return static response as a web service.

Here we are testing users, not testing orders developed by another user.

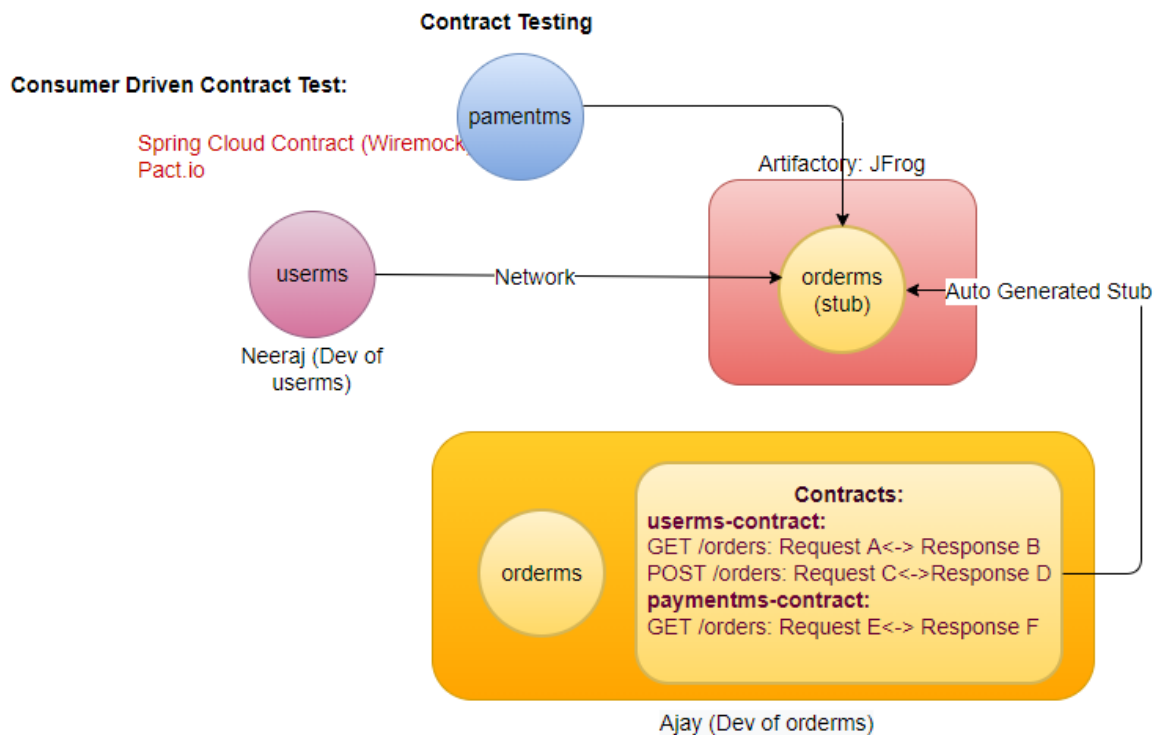
never do the changes in the older version, any changes will publish new version

Is preferred approach. **How to solve Issue of mismatch between stub and actual service** (using the contract service), will use the stub but with different way in contract testing.

Contract-Testing: (Extension of Integration Testing)

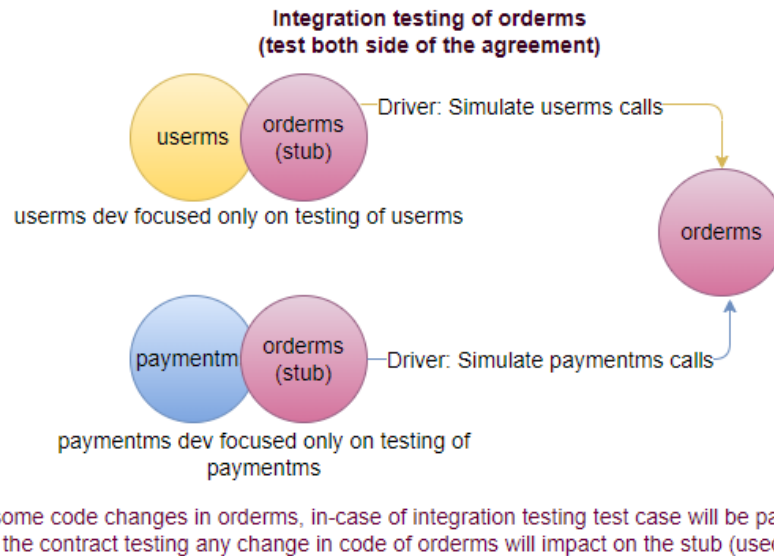


UC: orderms is used by users as well as paymentms, for the same ms having two different contracts, stub is generated based on the contract (separate contract for each client). **Contract is written, where API lives**, with the orderms, have different contract for different microservices (users, paymentms), based on the contract orderms generates the stubs.



Contract is autogenerated using the DSL (Domain Specific Language).

When usersms runs its case, will run with the stub, but orderms also needs to run their test with same stub, based on the same stub both microservices do their contract testing. In contract we just define our expectation



In this testing we don't do the testing of either orderms or testing of usersms, we do the testing of the contract (req<>res).

It can consider as extension of integration testing, its more robust, also support messaging solutions.

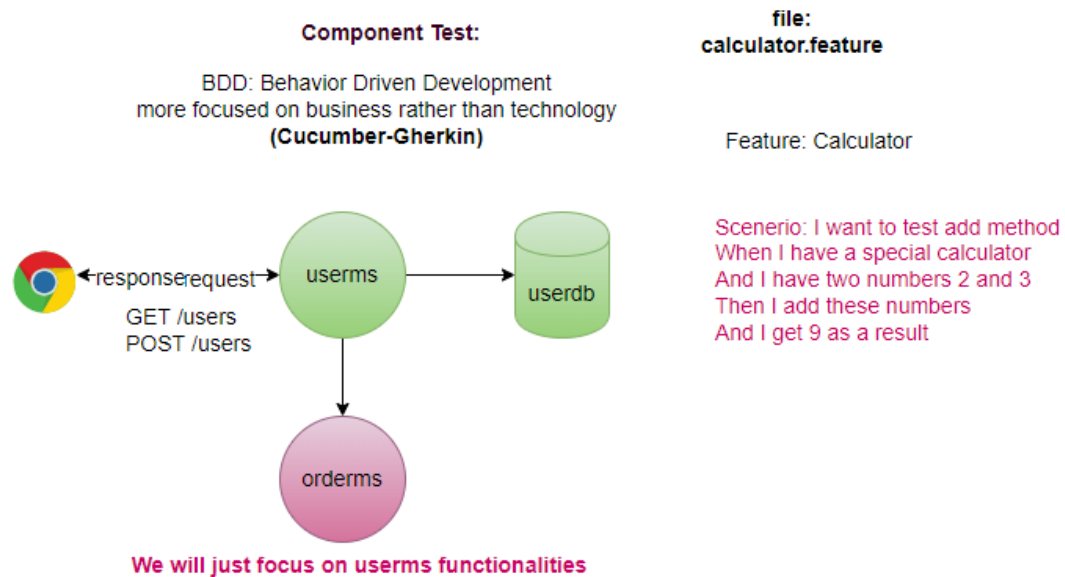
Component Testing: Test the whole component

Unit Test: Business Layer

Integration Test: Integration point (DB, other ms call)

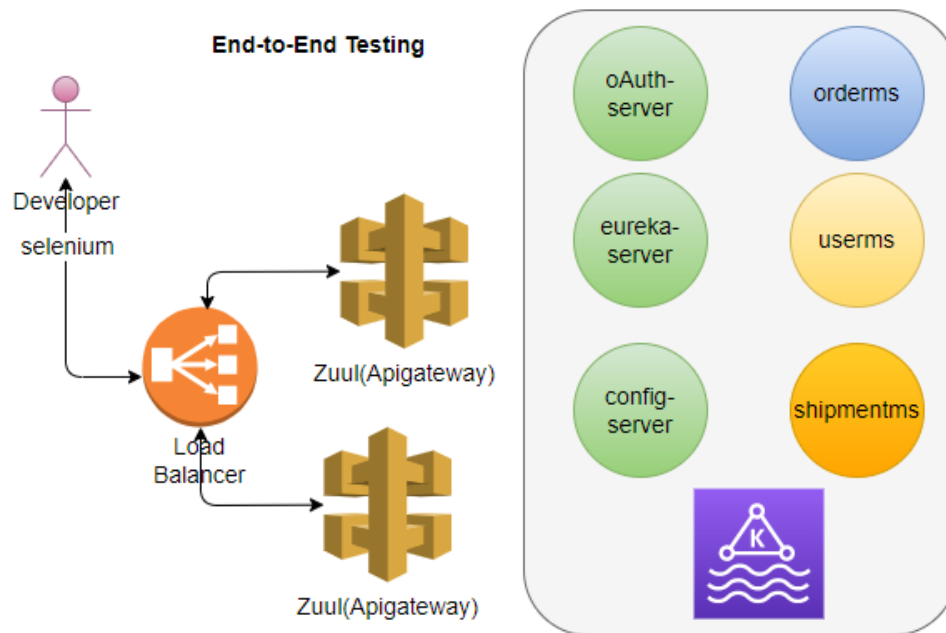
Contract Test: excluded the DB, focused on other service call.

Component Test: Whole ms, not specific point.



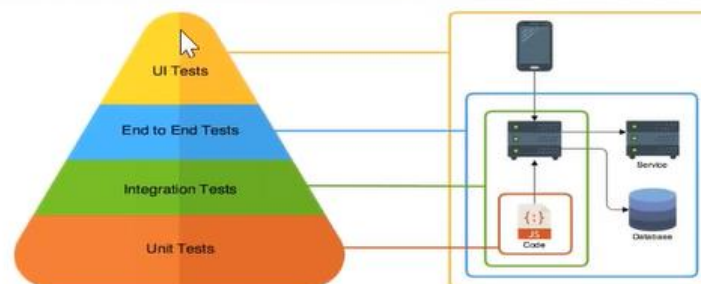
- Whole component as a black box
- Testing of the features as a whole, not an individual class.
- Don't care internally calls other service or DB.

End-to-End Testing



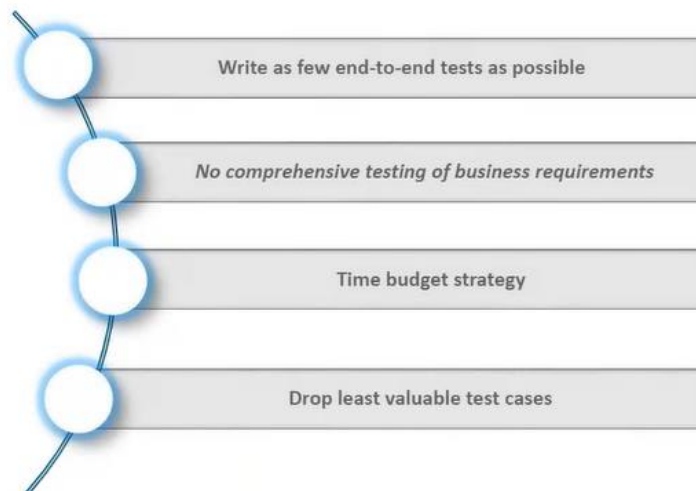
End to End Microservices Test

microservices architecture is built with multiple smaller parts that are choreographed dynamically, end-to-end tests provide value by adding coverage of the gaps between the services.



This ensures confidence in the correctness of messages passing between the services and verifies that the network infrastructure is correctly configured.

End to End Test Guidelines



- requires everything (UI and other components of the application) to be on place.
- UI included, use selenium to automate.

Generally, we run all tests (Unit, Integration, Component, Contract) separately with in the same pipeline, execute end-to-end tests in separate pipeline.

Testing Tools (part of Pipeline):

- Fortify (**Security Vulnerability** in your code)
- Black Duck (**Security Vulnerability** in third party code)
- IDE Plugin (Eclipse: EcEmma, Maven: Jacoco, SonarLint)- Line Coverage
- **SonarQube** (Line Coverage + Conditional Coverage + Code Smells)
- Mutation Test (Testing of Tests)-PIT (pitest.org)
- Chaos Engineering

DevOps Tools:

- Code Version: Git
- Configuration Management: Chef, Puppet, Ansible
- Containerization: Docker
- Container Orchestration: Kubernetes
- CI/CD Pipeline: Jenkins